

Laboratorio 1: Introducción a ANTLR

Análisis de la gramática MiniLang y el Driver.py

1. ¿Qué es un archivo .g4?

Un archivo .g4 es el archivo de gramática que utiliza ANTLR para generar automáticamente un analizador léxico (Lexer) y un analizador sintáctico (Parser) para un lenguaje definido por el programador. En este laboratorio, el archivo MiniLang.g4 define un pequeño lenguaje de tipo calculadora con variables, y contiene las siguientes secciones principales:

- **Declaración de la gramática:** la línea "grammar MiniLang;" le da nombre a la gramática. Este nombre debe coincidir exactamente con el nombre del archivo.
- **Reglas del Parser (minúsculas):** reglas como prog, stat y expr, que definen cómo se combinan los tokens para formar sentencias y expresiones válidas del lenguaje.
- **Reglas del Lexer (mayúsculas):** reglas como MUL, DIV, ID, INT y NEWLINE, que definen cómo se forman los tokens individuales a partir de los caracteres del texto de entrada.

2. La regla prog (punto de entrada)

La regla prog: stat+ ; es el punto de partida de la gramática. Indica que un programa válido en MiniLang está compuesto por una o más sentencias (stat) escritas de forma consecutiva. Esta es la regla que se invoca desde el Driver.py mediante parser.prog() para iniciar el análisis.

3. La regla stat y el uso de

La regla stat define tres tipos posibles de sentencia: una expresión suelta seguida de salto de línea, una asignación de variable, o una línea en blanco. El símbolo # que aparece después de cada alternativa (por ejemplo # printExpr, # assign, # blank) sirve para etiquetar cada alternativa con un nombre. ANTLR utiliza estas etiquetas para generar un método o clase independiente por cada alternativa dentro del código del Parser (y del Listener/Visitor), lo que permite distinguir programáticamente qué tipo de sentencia se encontró al recorrer el árbol sintáctico, en lugar de tener que revisar manualmente el contenido de cada nodo.

4. La regla expr y la precedencia de operadores

La regla `expr` define las expresiones matemáticas de forma recursiva: una expresión puede contener otras expresiones dentro de sí misma. Las alternativas incluyen multiplicación y división, suma y resta, un número entero, un identificador, o una expresión entre paréntesis. Un detalle importante de ANTLR es que el orden en que se escriben las alternativas determina su precedencia: como `MulDiv` aparece antes que `AddSub`, ANTLR le da mayor prioridad, resolviendo así la ambigüedad de expresiones como $2 + 3 * 4$ (se multiplica primero, como es matemáticamente correcto).

5. Reglas léxicas (tokens)

Las reglas en mayúsculas definen cómo se reconocen los tokens a partir del texto crudo de entrada. `ID` reconoce una o más letras consecutivas (nombres de variable), `INT` reconoce uno o más dígitos, y `NEWLINE` reconoce el salto de línea que marca el fin de una sentencia. La regla `WS` (espacios y tabulaciones) utiliza la instrucción `-> skip`, que le indica a ANTLR que descarte esos caracteres sin generar ningún token, ya que no aportan significado a la gramática.

6. Funcionamiento del Driver.py

El archivo `Driver.py` es el punto de entrada del programa en Python y conecta el Lexer y el Parser generados por ANTLR. Su funcionamiento sigue cuatro pasos:

- **FileStream:** lee el archivo de entrada (por ejemplo `program_test.txt`) como un flujo de caracteres.
- **MiniLangLexer:** toma ese flujo de caracteres y lo convierte en una secuencia de tokens, según las reglas léxicas definidas en la gramática.
- **CommonTokenStream:** envuelve los tokens generados por el Lexer en un flujo que el Parser puede consumir.
- **MiniLangParser y parser.prog():** el Parser toma el flujo de tokens y aplica las reglas sintácticas comenzando por la regla inicial `prog`, construyendo (o validando) el árbol sintáctico del programa. Si la entrada no respeta la gramática, ANTLR reporta los errores léxicos o sintácticos correspondientes en la consola; si es válida, el programa termina sin mostrar ningún mensaje.

7. Conclusión

En conjunto, la gramática `MiniLang.g4` y el `Driver.py` muestran el flujo básico de un compilador o intérprete: primero un análisis léxico que agrupa caracteres en tokens con significado, y luego un análisis sintáctico que valida que esos tokens sigan una estructura gramatical correcta. ANTLR automatiza la generación de ambos

componentes a partir de una única definición declarativa, lo que permite enfocarse en diseñar las reglas del lenguaje en lugar de implementar manualmente el analizador.

Enlaces

Video (YouTube, no listado): <https://youtu.be/YBo3vtLCF3s>

Repositorio de GitHub: <https://github.com/ecarcamo/CC3032-COMPIS>